

EXAMEN D'ALGO-SDA - L3I

Exercice 1 : listes doublement chaînées (6.25 Pts)

Un polynôme est constitué d'une somme de **monômes non nuls**. Chaque monôme est représenté par une structure composée de deux membres : degré (un entier) et coefficient (un réel).

Les polynômes seront représentés par une **liste doublement chaînée** de monômes non nuls (i.e : chaque cellule de la liste représente un monôme de coefficients non nuls) et **classés dans l'ordre croissant selon le degré**.

Pour faciliter la gestion des polynômes, on rajoutera deux adresses : **la première adresse pour indiquer la tête de la liste et la seconde adresse pour indiquer la queue de la liste**.

La liste doublement chaînée représentant le polynôme est définie comme suit :

<pre>typedef struct{ double coef ; int degre ; } Monome ; typedef struct { DListe tete; DListe queue; } Polynome;</pre>	<pre>typedef struct cellule{ Monome monome ; struct cellule* suivant ; struct cellule* precedent ; }*DListe ;</pre>
---	---

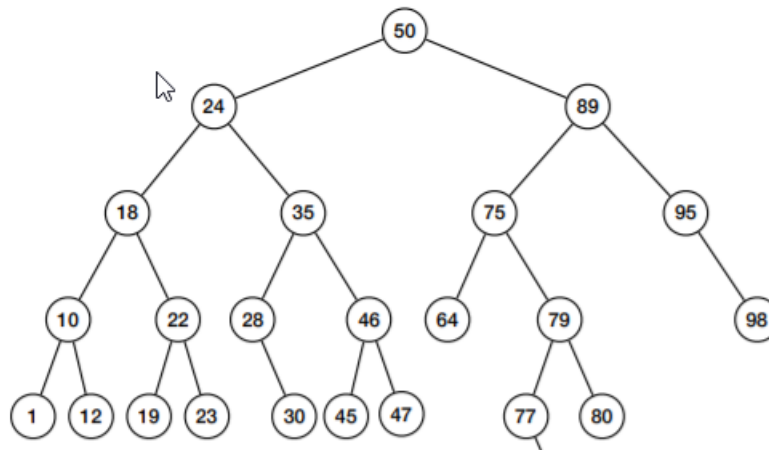
- 1) **Ecrire en C**, l'algorithme d'insertion en tête d'un monôme donné dans un polynôme donné.
- 2) **Ecrire en C**, l'algorithme d'insertion en queue d'un monôme donné dans un polynôme donné.
- 3) **Ecrire en C**, l'algorithme de suppression en tête d'un monôme donné dans un polynôme donné.
- 4) **Ecrire en C**, l'algorithme de suppression en queue d'un monôme donné dans un polynôme donné.
- 5) Ecrire en langage algorithmique ou en C, un algorithme qui calcule le polynôme représentant l'intersection de deux polynômes donnés selon le schéma suivant :

$$P1(x) = 3 + 5x^2 + 6x^3 - 2x^4 - 8x^5, P2(x) = 7x + 4x^2 - 6x^3 + 7x^5 - 9x^8, \quad P1(x) \cap P2(x) = P(x) = 9x^2 - x^5$$

Note : P1 et P2 ne seront pas modifiés après l'intersection.

- 6) En parcourant le polynôme dans les deux sens ; écrire en langage algorithmique (ou C), un algorithme qui détermine l'adresse d'un monôme donné dans un polynôme donné s'elle existe et NULL s'elle n'existe pas.
- 7) En utilisant la question précédente ; écrire en langage algorithmique (ou C), un algorithme qui supprime un monôme donné dans un polynôme donné.

Exercice 2 ABR & AVL (6.0 Pts)

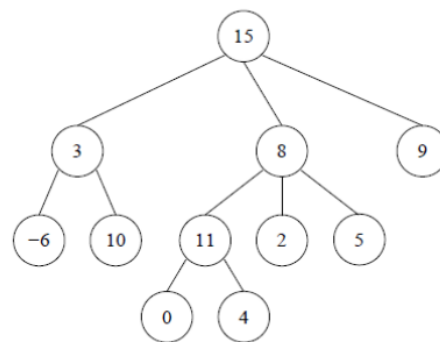


- 1) Décrire le principe de l'algorithme d'insertion racine dans un ABR donné.
- 2) Appliquer votre algorithme d'insertion racine pour l'insertion de 78 dans l'ABR ci-dessus.
- 3) Décrire le principe de l'algorithme de suppression racine dans un ABR donné.
- 4) Appliquer votre algorithme de suppression racine à l'ABR ci-dessus.
- 5) Insérer 78 dans l'AVL ci-dessus.
- 6) Supprimer 75 dans l'AVL ci-dessus.
- 7) **Ecrire en C**, la structure de donnée représentant un AVL, dont le membre donné est de type TElement (Entier par exemple).
- 8) **Ecrire en C**, l'algorithme qui effectue la rotation gauche dans un AVL donné.
- 9) **Ecrire en C**, l'algorithme qui effectue la rotation droite dans un AVL donné.

Exercice 3 : Arbres généraux (3.75 PTS)

Lorsque le nombre de fils n'est pas connu a priori, on utilise une liste chaînée de fils pour implémenter les arbres généraux. Voici la structure :

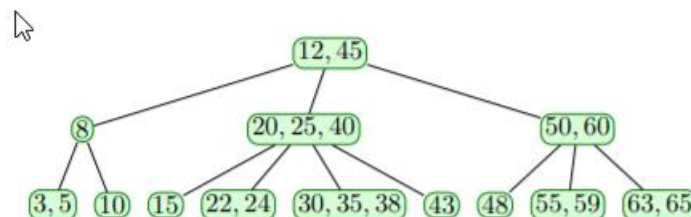
```
typedef struct noeud * ArbreG;
typedef struct cellule * ListArbres ;
struct noeud{
    TElement donnee;
    ListArbres fils ;
};
struct cellule{
    ArbreG premierfils;
    ListArbres suivant;
};
```



- 1) En respectant la structure ci-dessus, représenter graphiquement l'arbre ci-dessus.
- 2) Ecrire en langage algorithmique ou en C, un algorithme qui compte le nombre de feuilles présente dans un arbre donné.
- 3) Le degré d'un nœud est le nombre des fils du nœud.
Le degré d'un arbre est égal au plus grand des degrés de ses nœuds :

$$d^{\circ}(\text{Arbre}) = \max \{ d^{\circ}(nd) \mid nd \text{ est un nœud de l'Arbre} \}$$
Ecrire en langage algorithmique ou en C, un algorithme qui détermine le degré d'un arbre donné.

Exercice 4 : Arbre 2-3-4 (4.0 PTS)



- 1) Insérer l'élément 37 dans l'arbre 2-3-4 ci-dessous.
- 2) On suppose que les nœuds de l'arbre 2-3-4 sont des 4-Nœuds. **Ecrire en C**, les structures de données nécessaires pour représenter les arbres 2-3-4 dont le membre donné est de type TElement (Entier par exemple).
- 3) **Ecrire en C**, le parcours en largeur d'un arbre 2-3-4 donné.
- 4) **Ecrire en C**, un algorithme qui retourne à la fois l'adresse du minimum et l'adresse du maximum dans un arbre 2-3-4 donné.

Note : On suppose connu, les opérations de la structure intermédiaire utilisée.

NOTES

- ♦ Les réponses aux questions doivent être claires et justifiées.
- ♦ Pour chaque algorithme (ou fonction en C), vous aurez soin d'expliquer les différentes actions de l'algorithme les préconditions, ainsi que le passage des paramètres.

♦ **Chaque algorithme (ou fonction en C) supplémentaire utilisé doit être défini.**

*****Annexes*****

On suppose connu les algorithmes suivants :

Structure Polynôme :

```

Fonction degre : Monome → Entier
Fonction coef : Monome → Reel
Fonction donnee : DListe → Monome
Fonction suivant : DListe → DListe
Fonction precedent : DListe → DListe
Fonction initDL : [] → DListe
Fonction estVideDL : DListe → Booléenne
Fonction tete : Polynome → DListe
Fonction queue : Polynome → DListe
Fonction initP : [] → Polynome
Fonction estVideP : Polynome → Booléenne

```

Structure AVL :

```

Fonction hauteur : AVL → Entier
Fonction donnee : AVL → TElement
Fonction filsGauche : AVL → AVL
Fonction filsDroit : AVL → AVL
Fonction initA : [] → AVL
Fonction estVideA : AVL → Booléenne

```

Structure Arbres généraux

```

Fonction donnee : ArbreG → TElement
Fonction fils : ArbreG → ListArbres
Fonction premierFils : ListArbres → ArbreG
Fonction suivant ListArbres → ListArbres
Fonction initAG : [] → ArbreG
Fonction estVideAG : ArbreG → Booléenne

```

Structure Arbres234

```

Fonction nbFils : Arbres234 → Entier
Fonction iEmeDonnee: Entier x Arbres234 → TElement
Fonction iEmeFils: Entier x Arbres234 → Arbre234

```

Structure File

```

Fonction sommetF : File → TElement
Fonction enfiler : TElement x File → File
Fonction defiler : File → File
Fonction initF : [] → File
Fonction estVideF : File → Booléenne
Fonction estPleinF : File → Booléenne

```

Structure Pile

```

Fonction sommetP : Pile → TElement
Fonction empiler : TElement x Pile → Pile
Fonction depiler : Pile → Pile
Fonction initP : [] → Pile
Fonction estVideP : Pile → Booléenne
Fonction estPleinP : Pile → Booléenne

```